

PROJECT: EASYSHARE
SECURITY DOCUMENTATION



Security Documentation

for the

EasyShare System

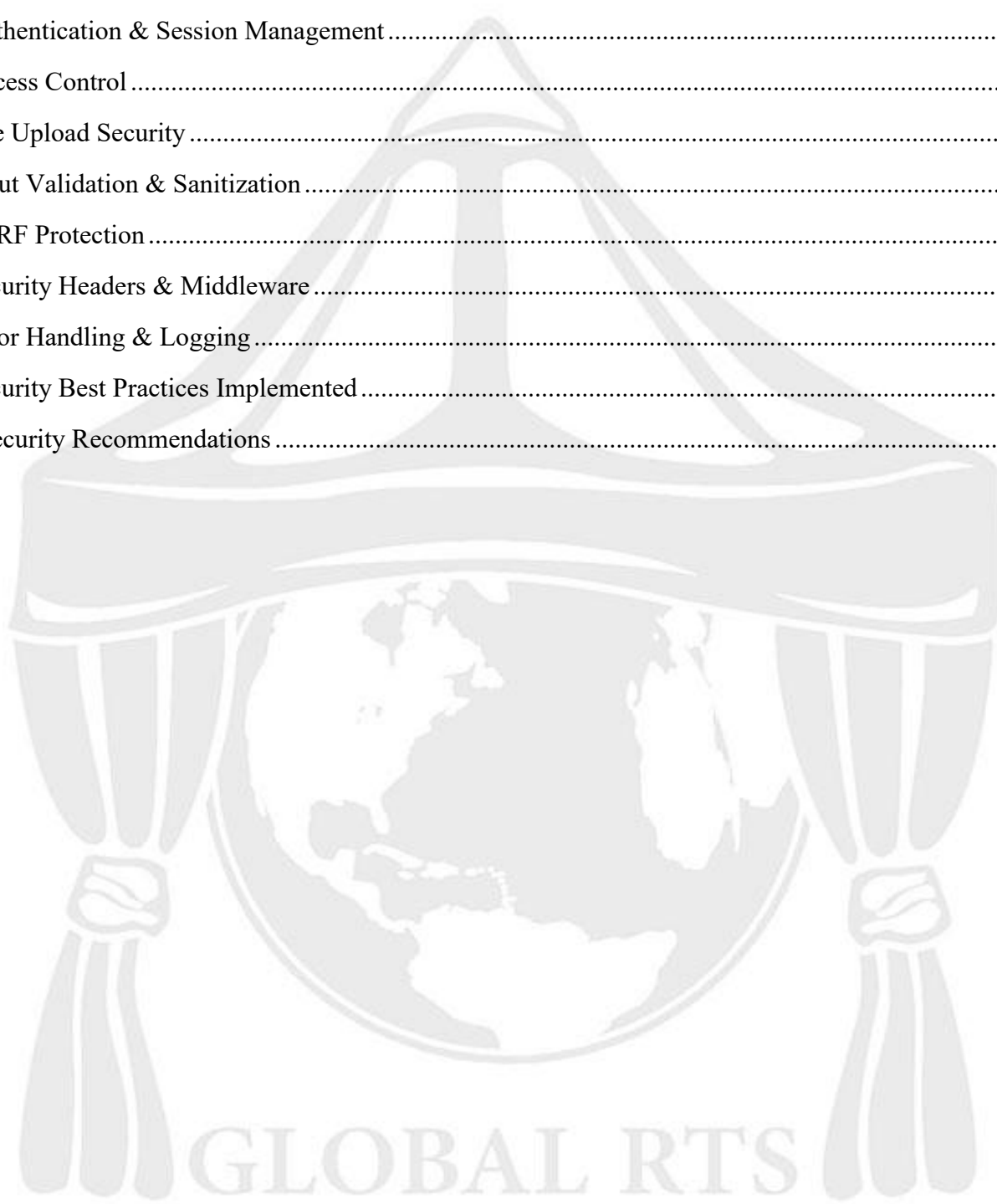
GLOBAL RTS

PROJECT: EASYSHARE SECURITY DOCUMENTATION



Content

1. Overview	3
2. Authentication & Session Management	3
3. Access Control	3
4. File Upload Security	5
5. Input Validation & Sanitization	5
6. CSRF Protection	5
7. Security Headers & Middleware	6
8. Error Handling & Logging	7
9. Security Best Practices Implemented	7
10. Security Recommendations	8



1. Overview

EasyShare is a secure, web-based file management system. It leverages multiple layers of security including session protection, CSRF, input validation, and hardened HTTP headers.

2. Authentication & Session Management

- Sessions are managed with express-session
- Cookies are:
 - ❖ *httpOnly: true*
 - ❖ *sameSite: 'strict'*
 - ❖ *secure:false*: Set to *true* in production.
- Trust the first proxy: Required for secure cookies behind proxies

```
this.app.set('trust proxy', 1);
```

3. Access Control

- Authenticated routes protected with middleware.

```
isLoggedIn(req, res, next) {  
    if (req.session && req.session.username) {  
        return next();  
    } else {  
        return res.redirect('/login'); }  
}
```

PROJECT: EASYSHARE SECURITY DOCUMENTATION



- Role-based access used to restrict admin functions.
- Login with 2FA: The logic regarding login and 2FA is described in the methods

- ❖ */login*

- ❖ */2fa-verified*

- ❖ */verify*

- Middleware protection against Brute Force

- ❖ Track failed login attempts

```
const failedLoginAttempts = {};  
  
const MAX_FAILED_ATTEMPTS = 10;  
  
const BLOCK_TIME = 5 * 60 * 1000;
```

- ❖ Check if the IP or user is blocked due to too many failed login attempts

```
isBlocked(username) {  
  
    const record = failedLoginAttempts[username];  
  
    if (record) {  
  
        const { failedAttempts, blockExpiresAt } = record;  
  
        if (failedAttempts >= MAX_FAILED_ATTEMPTS &&  
blockExpiresAt > Date.now()) {  
  
            return true;  
  
        }  
  
        if (blockExpiresAt && blockExpiresAt <= Date.now()) {  
  
            failedLoginAttempts[username] = { failedAttempts: 0 };  
  
        }  
  
    }  
  
}
```



```
    }  
  
    }  
  
    return false;  
}
```

4. File Upload Security

- Multer (memory storage) ensures temporary handling of uploads.
- **Recommendation:** Validate MIME types, limit size and signatures of all upload files.

5. Input Validation & Sanitization

- Input limits configured:

```
this.app.use(express.json({ limit: '10mb' }));  
  
this.app.use(express.urlencoded({ limit: '10mb', extended: true }));
```

- Add validation logic with libraries like *Joi*

6. CSRF Protection

```
this.app.use(csurf({ cookie: false }));  
  
this.app.use((req, res, next) => {  
    res.locals.csrfToken = req.csrfToken();  
  
    next();  
});  
  
this.app.get('/csrf-token', (req, res) => {  
    res.json({ csrfToken: req.csrfToken() }); });
```



7. Security Headers & Middleware

- Helmet Middleware

```
this.app.use(helmet());
```

- Prevent HTTP Parameter Pollution

```
this.app.use(hpp());
```

- Custom Security Headers

```
this.app.use((req, res, next) => {  
  
    res.setHeader('Content-Security-Policy', "default-src 'self';  
img-src 'self' data:; script-src 'self'; style-src 'self' 'unsafe-  
inline';");  
  
    res.setHeader('X-Content-Type-Options', 'nosniff');  
  
    res.setHeader('Cache-Control', 'no-store, no-cache, must-  
revalidate, private');  
  
    res.setHeader('Strict-Transport-Security', 'max-age=31536000;  
includeSubDomains; preload');  
  
    res.setHeader('X-Frame-Options', 'DENY');  
  
    res.setHeader('Referrer-Policy', 'no-referrer');  
  
    res.setHeader('Permissions-  
Policy', "geolocation=(),microphone=(),camera=()");  
  
    next();  
  
});
```

- Error Handling & Logging

PROJECT: EASYSHARE

SECURITY DOCUMENTATION



```
this.app.use((err, req, res, next) => {  
  
    console.error(err.stack);  
  
    if (err.code === 'EBADCSRFTOKEN') {  
  
        return res.status(403).send('Form tampered with!');  
  
    }  
  
    res.status(500).send('Something broke!');  
  
});
```

8. Error Handling & Logging

```
this.app.use((err, req, res, next) => {  
  
    console.error(err.stack);  
  
    if (err.code === 'EBADCSRFTOKEN') {  
  
        return res.status(403).send('Form tampered with!');  
  
    }  
  
    res.status(500).send('Something broke!');  
  
});
```

9. Security Best Practices Implemented

Feature	Status
Trust first proxy	✓ Configured
Session hardening	✓ Enabled
CSRF protection	✓ Enabled
Helmet for secure headers	✓ Enabled
HTTP Parameter Pollution (HPP)	✓ Protected
Custom security headers	✓ Configured
Body size limits	✓ Enabled
Global error handling	✓ Enabled

PROJECT: EASYSHARE SECURITY DOCUMENTATION



Table 1: Security best practices

10. Security Recommendations

- Already configured:
 - ❖ Trust proxy, secure headers, CSRF, HPP, and error handling.
- Next steps:
 - ❖ Enable secure: true in session cookies for production.
 - ❖ Implement rate limiting (express-rate-limit).
 - ❖ Validate and sanitize all user input.
 - ❖ Log suspicious activity to a secure location.